Use Case: Check Game End (Win/Lose/Tie)

Iteration: 1

Primary Actor: System

Goal in Context: Determine if the game has reached an end state after each move, based on win, lose, or tie conditions.

Preconditions:

1. The game is in progress with an active session.
2. A player has completed their move and placed a piece on the board.
3. The system has updated the game state with the latest move.
4. Conditions for winning, losing, or a tie are defined and accessible in the game logic (e.g., three in a row for Tic-Tac-Toe, four in a row for Connect 4, or no moves left in Checkers).

Trigger: A player completes a move.

Scenario:

1. The system updates the board with the player's move.
2. The system evaluates the board state to determine if any end-game conditions are met:
    - **Tic-Tac-Toe**:
        - The system checks for three identical pieces in a row, column, or diagonal to identify a win.
        - The system declares a tie if the board is full and no winning condition is met.
    - **Checkers**:
        - The system checks if one player has no remaining pieces or valid moves, declaring the opponent the winner.
    - **Connect 4**:
        - The system checks for four identical pieces in a row, column, or diagonal to determine a win.
        - The system declares a tie if all columns are filled and no win condition is met.
3. If an end-game condition is met, the system concludes the game and declares the result (win, lose, or tie).
4. The system notifies both players of the game outcome.

Postconditions:

1. If a win, lose, or tie condition is met, the game session is ended and the result is recorded.
2. Else no end-game condition is met and the next turn continues.

Exceptions:

- If an error occurs in evaluating the board state (e.g., system glitch), the system resets the board to its last valid state and prompts the player to attempt the same or new move again.

Priority: High, as identifying game-end conditions is crucial for determining the game's outcome.

When Available: 1st iteration.

Frequency of Use: Once per game at the end of the game.

Channel to Actor: The system automatically verifies players via the game interface.

Secondary Actors: Both players in the game session.

Channels to Secondary Actors: The result is displayed on each player's screen at the end of the game.

Open Issues: None.

Use case: End Game

Iteration: 1

Primary actor: Player

Goal in context: Notify the network that the game has ended.

Preconditions: A game is currently in session

Trigger: A final move is made by the player that terminates the game.

Scenario:

1. Starts a game
2. Several moves are made by player(s)
3. A move is made by a player that ends the game
4. Network is notified that the game has ended due to a terminating play

Postconditions: Data is cleaned and reset for next game.

Exceptions: N/A

Priority: High priority

Frequency of use: High frequency

Channel to actor: Mouse

Secondary actors: Network

Channels to secondary actors: Server

Open Issues: N/A

Use Case: Place Piece

Iteration: 1

Primary Actor: Player

Goal in Context: Allow the player to place a game piece on the board during their turn.

Preconditions:

1. The player is logged into their account.
2. The player is connected to an active game session.
3. It is the player's turn to make a move.
4. The player has selected a specific grid location on the game board.

Trigger: The player decides to place a piece on the game board during their turn.

Scenario:

1. The player selects an available spot on the game board by clicking or tapping.
2. The system does a verification check that it is the player's turn.
3. The system checks if the move is valid.
4. The system processes the piece placement based on the game type:
    a. Tic-Tac-Toe
        i. The player selects a square on the 3x3 grid.
    b. Connect 4
        i. The player selects a column in the grid.
        ii. The system drops the piece to the lowest available row in that column.
    c. Checkers
        i. The player selects a valid move for their checker piece, following standard movement rules.
        ii. The system verifies for a capture
5. The system updates the game state and checks for any winning or game-ending conditions.
6. The system notifies the opponent that it is now their turn.

Post Conditions:

1. The game board is updated with the newly placed piece.
2. If a winning condition is met, the game concludes and a winner is declared.
3. Else the turn is handed over to the opponent.

Exceptions:

- The player attempts to place a piece outside the boundaries of the board.
- The player selects an occupied spot (e.g., in Tic-Tac-Toe).
- The player makes an invalid move for the selected piece (e.g., in Checkers, pieces can only move diagonally on dark squares).

Priority: High, as placing a piece is a core action in each game.

When available: 1st iteration

Frequency of Use: Multiple times per game, depending on game length.

Channel to Actor: The player interacts through the screen by tapping or clicking on the board.

Secondary Actors: Opponent player.

Channels to Secondary Actors: Updates appear on the opponent's screen to reflect the placed piece and turn change.

Open Issues: None.

Use case: Start Game

Iteration: 1

Primary actor: Player

Goal in context: Customer starts a game of Tic-Tac-Toe, Checkers or Connect 4.

Preconditions: Player has already selected the game they would like to start.

Trigger: Player clicks the start icon/button on screen.

Scenario:

1. Player opens application
2. Player selects a game they want to play
3. Player clicks start

Postconditions: Game commences, and the player can make a move.

Exceptions: N/A

Priority: High priority

Frequency of use: High frequency

Channel to actor: Mouse

Secondary actors: N/A

Channels to secondary actors: N/A

Open Issues:

- The game the player is interested in may not be available
- The player may want to continue an existing game

Use Case: Update Player Timer

Iteration: 3

Primary Actor: System

Goal in Context: Update the respective timer for each player

Preconditions:

1. A game session has started.
2. The first player makes a move.

Trigger: Either when a game starts, or a player makes a move and changes the turn to the other player.

Scenario:

1. After the game starts, the timer for the first player starts going down.
2. If the first player makes a move, the system:
    - Stops the countdown for the first player timer
    - Stores the timer for Player1
    - Starts the countdown for the second player timer
3. If either of the timers reach an end, the system notifies a corresponding winning message (one player wins).

Postconditions:

1. If the timer ends, the system is notified and the game ends with the corresponding winner being displayed
2. Otherwise, if both timers are still running, the game continues normally.

Exceptions: N/A

Priority: Medium, as the game can work without such functionality. This would help to

increase the complexity of the games, but it’s not mandatory for testing.

When Available: 3rd iteration.

Frequency of Use: Multiple times throughout the game.

Channel to Actor: The system checks both timers’ status automatically.

Secondary Actors: Both players in the game session.

Channels to Secondary Actors: Both players’ respective timer are included in the visual interface

Open Issues: None.

Use case: Resign Game

iteration: 1

primary actor: Player

Goal in context: The player resigns, resulting in a loss and concluding the game.

Precondition: The player is currently in an active game.

Trigger: The player wishes to conclude the game immediately.

Scenario:
1. Player wishes to end the current game
2. Player pressed the resign game button
3. game ends and player loses

Post conditions: The game ends and the player loses

Exceptions:
1. The resignation button malfunctions
2. network issue or player disconnects prior to resigning
3. player attempts to end game after game has already ended

Priority: High

When available: 1 iteration

Frequency of use: infrequent - Expected when players want to end the game prematurely and lose.

Channel to actor: Player's finger has physical interaction with the resign game button

Secondary actors: System

Channel to secondary actors: System receives feedback that game wants to be ended

Open issues:

- **How to handle user misinput:** Should we consider adding a confirmation pop-up to avoid unintended resignations?

Use case: Choose a Side

iteration: 1

primary actor: Player

Goal in context: The player chooses a side.

Precondition: The player started a game.

Trigger: The player starts a game and wants to pick a side.

Scenario:

1. Player starts a game
2. player selects colour or shape from available colours or shapes
3. game begins and other player moves

Post conditions: The game begins and other player moves

Exceptions:

1. The colour or shape malfunctions
2. The game times out

Priority: High

When available: 1 iteration

Frequency of use: Frequent - Every time a new game is started

Channel to actor: Player's finger has physical interaction with the screen to select their side

Secondary actors: System

Channel to secondary actors: System receives feedback to begin game

Open issues:

- First move advantage: How can we ensure an equal share of first moves after players choose their sides?

Use case: Check for Captures (Checkers)

iteration: 1

primary actor: System

Goal in context: The System checks for captures

Precondition: A move has been made

Trigger: A player moves

Scenario:

1. Players moves their piece (either jumps or does not)
2. system checks if a piece has been captured
3. System returns true if a capture occurred, otherwise false

Post conditions: The system accurately returns true or false based on the capture status of the move.

Exceptions:

1. System times out

Priority: High

When available: 1 iteration

Frequency of use: Frequent - Every time a player moves

Channel to actor: System receives feedback signalling a move has been made

Secondary actors: N/A

Channel to secondary actors: N/A

Open issues:

- Multiple captures: How should we check for multiple captures? Is it different from one capture?

Use Case: Validate Move

Iteration: 1

Primary Actor: System

Goal in Context: Ensure that a player's move is valid according to the rules of the game.

Preconditions:

1. The player is logged into their account.
2. The player is connected to an active game session.
3. It is a player's turn to make a move.
4. The player has selected a specific grid location on the game board.
5. The selected move complies with the basic rules of the game (e.g., selecting an empty cell in Tic-Tac-Toe or a valid jump in Checkers).

Trigger: A player attempts to place a piece or make a move on the board.

Scenario:

1. A player selects a location on the board to place a piece or move a piece.
2. The system verifies that it is the player's turn.
3. The system checks that the selected move follows the specific game rules:
    a. Tic-Tac-Toe:
        i. The system ensures the selected cell is unoccupied.
    b. Checkers:
        i. The system confirms that the move is within the piece's movement range (diagonal move for regular pieces, any direction for kings).
        ii. The system checks if the move is a valid capture, if applicable.
    c. Connect 4:
        i. The system verifies that the selected column has an open spot in the lowest available row.
4. If the move is valid, the system allows the move to proceed.
5. If the move is invalid, the system displays an error message and prompts the player to select a valid move.

Postconditions:

1. The move is validated and, if valid, will be executed on the board.
2. If the move is invalid, the system prevents the action and requests the player to make another selection.

Exceptions:

- A player attempts to place a piece outside the boundaries of the board.
- A player selects an occupied spot (e.g., in Tic-Tac-Toe).
- A player makes an invalid move for the selected piece (e.g., in Checkers, pieces can only move diagonally on dark squares).

Priority: High, as move validation is essential to prevent invalid actions in each game.

When Available: 1st iteration.

Frequency of Use: Multiple times per game, depending on game length.

Channel to Actor: The system automatically checks and notifies players via the game interface.

Secondary Actors: Both players in the game session.

Channels to Secondary Actors: If the move is valid, the updated board state is displayed on each player’s screen

Open Issues: None.